Programação Web

ORM, Query Builders e Scope

Prof. Diego Cardoso Borda Castro

Organização de código

- Camadas de abstração
  - Index
  - Controller
    - Router
  - DTO
  - Service
  - Repositório
  - Entidade

Acesso ao banco

- Manipulação de dados em banco
  - Query pura
    - MySql2
  - Query Builder
    - Maior flexibilidade
  - ORM
    - Organização
    - Entidades

	Gerência	Abstração	Complexidade
Raw	Alto		Baixo
Query builder	Baixo	Baixo	Baixo
ORM	Baixo	Alto	Média

Query builder

- Knex
  - https://knexjs.org/guide/

```
$ npm install knex --save

# Then add one of the following (adding a --save) flag:
$ npm install pg
$ npm install pg-native
$ npm install sqlite3
$ npm install better-sqlite3
$ npm install mysql
$ npm install mysql2
$ npm install oracledb
```

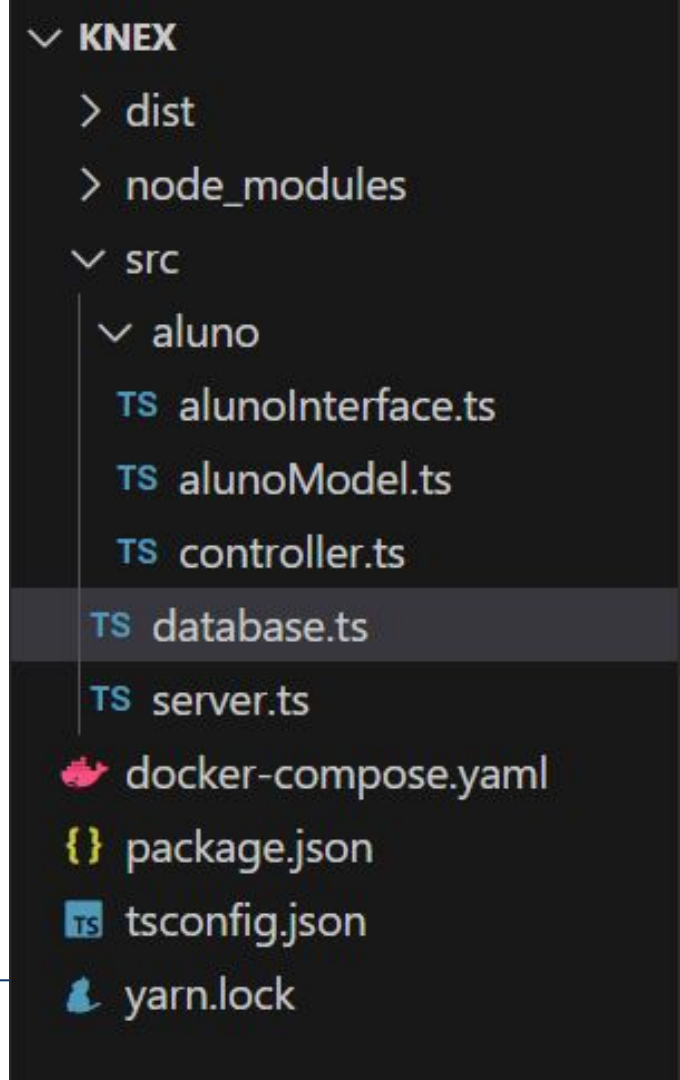
Knex

- Knex
  - Configuração básica
  - process.env.VAR

```
1  import Knex from 'knex';
2
3  export const database = Knex( {
4      client: 'postgresql',
5      connection: {
6          host: '127.0.0.1',
7          port : 15432,
8          database: 'postgres',
9          user: 'postgres',
10         password: 'postgres',
11     },
12     pool: {
13         min: 2,
14         max: 10,
15     },
16 });
```

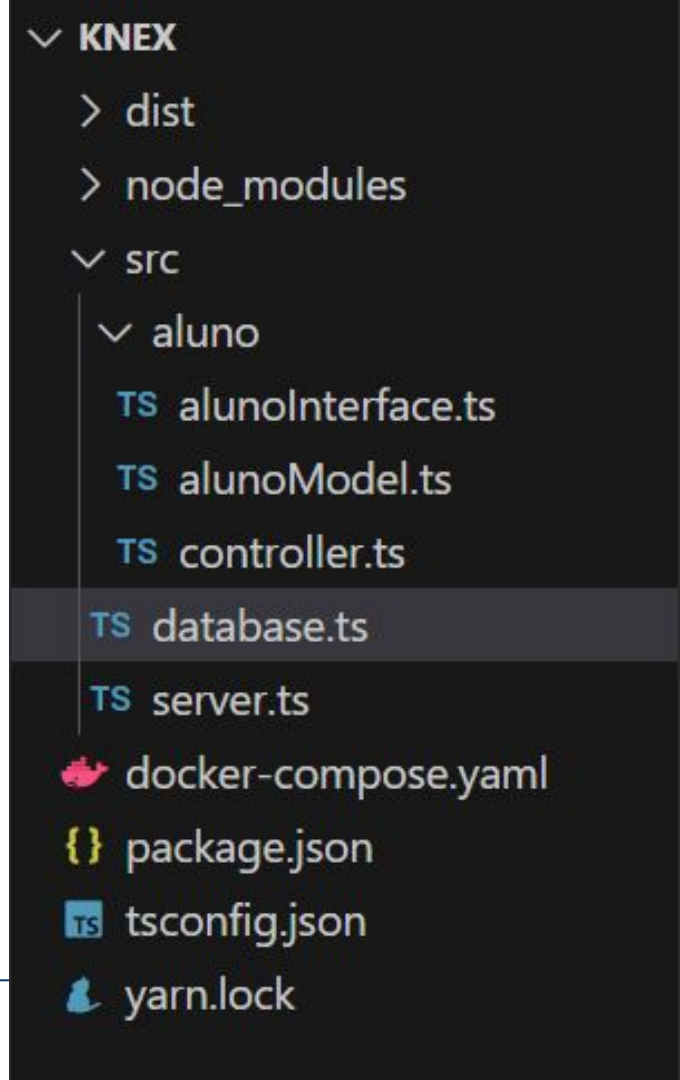
Knex

```import {Request, Response} from "express"
import alunoService from "./alunoModel"

export async function CriarAluno(request: Request, response: Response) {
    await alunoService.criar({
        nome: 'Diego',
        matricula: '123'
    });
}

export async function ListarAluno(request: Request, response: Response) {
    let alunos = await alunoService.listar();
}
```



Knex

```
1  import { database } from '../database';
2
3  class Aluno {
4
5      async criar({nome, matricula}: AlunoInterface){
6          await database('aluno').insert({
7              nome: nome,
8              matricula: matricula
9          });
10     }
11
12     async listar(){
13         return await database('aluno').select();
14     }
15 }
16
17 export default new Aluno();
```



Knex

- Select
- Insert
- Update
- Delete
- Join

```
knex.select('title', 'author', 'year')
  .from('books')

knex.select()
  .table('books')
```

```
knex.select(knex.ref('id').as('identifier'))
  .from<User>('users'); // Resolves to { identifier: number; }[]
```

Knex

- Select
- Insert
- Update
- Delete
- Join

```
knex.avg('sum_column1')
  .from(function() {
    this.sum('column1 as sum_column1')
      .from('t1')
      .groupBy('column1')
      .as('t1')
  })
  .as('ignored_alias')
```

Knex

- Select
  - .groupBy('id')
  - .orderBy('expires_at')
- Insert
- Update
- Delete
- Join

```
knex.select('*')
  .from('users')
  .limit(10)
  .offset(30)
```

```
knex.select('*')
  .from('users')
  .whereNull('last_name')
  .union(
    knex.raw(
      'select * from users where first_name is null'
    ),
    knex.raw(
      'select * from users where email is null'
    )
  )
```

Knex

- Select
- Insert
- Update
- Delete
- Join

```
knex('books').insert({title: 'Slaughterhouse Five'})

// Normalizes for empty keys on multi-row insert:
knex('coords').insert([{x: 20}, {y: 30},  {x: 10, y: 20}])

// Returns [2] in "mysql", "sqlite"; [2, 3] in "postgresql"
knex
  .insert(
    [
      { title: 'Great Gatsby' },
      { title: 'Fahrenheit 451' }
    ],
    ['id']
  )
  .into('books')
```

Knex

- Select
- Insert
  - merge
- Update
- Delete
- Join

```
knex('tableName')
  .insert({
    email: "ignore@example.com",
    name: "John Doe",
    active: true
  })
    // ignore only on email conflict and active is true.
  .onConflict(knex.raw('(email) where active'))
  .ignore()
```

Knex

- Select
- Insert
- Update
- Delete
- Join

```
knex('books')
  .where('published_date', '<', 2000)
  .update({
    status: 'archived',
    thisKeyIsSkipped: undefined
  })

// Returns [1] in "mysql", "sqlite", "oracle";
// [] in "postgresql"
// unless the 'returning' parameter is set.
knex('books').update('title', 'Slaughterhouse Five')

/** Returns
 * [{
 *   id: 42,
 *   title: "The Hitchhiker's Guide to the Galaxy"
 * }] **/
knex('books')
  .where({ id: 42 })
  .update({
    title: "The Hitchhiker's Guide to the Galaxy"
  }, ['id', 'title'])
```

Knex

- Select
- Insert
- Update
- Delete
- Join

```
knex('accounts')
  .update({ enabled: false })
  .updateFrom('clients')
  .where('accounts.id', '=', 'clients.id')
  .where('clients.active', '=', false)
```

Knex

- Select
- Insert
- Update
- Delete
- Join

```
// insert new row with unique index on title column
knex('books').upsert({title: 'Great Gatsby'})

// update row by unique title 'Great Gatsby'
// and insert row with title 'Fahrenheit 451'
knex('books').upsert([
  {title: 'Great Gatsby'},
  {title: 'Fahrenheit 451'}
], ['id'])

// Normalizes for empty keys on multi-row upsert,
// result sql:
// ("x", "y") values (20, default), (default, 30), (10, 20):
knex('coords').upsert([{x: 20}, {y: 30}, {x: 10, y: 20}])
```

Knex

- Select
- Insert
- Update
- Delete
- Join

```
knex('accounts')
  .where('activated', false)
  .del()
```

Knex

- Select
- Insert
- Update
- Delete
- Join

```
knex('users')
  .join('contacts', 'users.id', '=', 'contacts.user_id')
  .select('users.id', 'contacts.phone')

knex('users')
  .join('contacts', 'users.id', 'contacts.user_id')
  .select('users.id', 'contacts.phone')
```

```
knex.select('*').from('users').join('accounts', function() {
  this.on(function() {
    this.on('accounts.id', '=', 'users.account_id')
    this.orOn('accounts.owner_id', '=', 'users.id')
  })
})
```

Knex

- Select
- Insert
- Update
- Delete
- Join
  - Left
  - Right
  - …

```
knex
  .from('users')
  .innerJoin('accounts', 'users.id', 'accounts.user_id')

knex
  .table('users')
  .innerJoin(
    'accounts',
    'users.id',
    '=',
    'accounts.user_id'
  )

knex('users')
  .innerJoin('accounts', function() {
    this
      .on('accounts.id', '=', 'users.account_id')
      .orOn('accounts.owner_id', '=', 'users.id')
  })
```

Knex

- Outros métodos
  - .count()
  - .min()
  - .max()
  - .sum()
  - .avg()
  - .truncate()

```
knex('users').where('votes', '>', 100)

const subquery = knex('users')
  .where('votes', '>', 100)
  .andWhere('status', 'active')
  .orWhere('name', 'John')
  .select('id');

knex('accounts').where('id', 'in', subquery)
```

Knex

- Transaction
  - Bloco de código
  - Commit
  - Rollback

```
const Promise = require('bluebird');
knex.transaction(function(trx) {
  knex('books').transacting(trx).insert({name: 'Old Books'})
    .then(function(resp) {
      const id = resp[0];
      return someExternalMethod(id, trx);
    })
    .then(trx.commit)
    .catch(trx.rollback);
})
.then(function(resp) {
  console.log('Transaction complete.');
})
.catch(function(err) {
  console.error(err);
});
```

ORM

- Sequelize
  - https://sequelize.org/docs/v6/
  - https://github.com/sequelize/sequelize-typescript

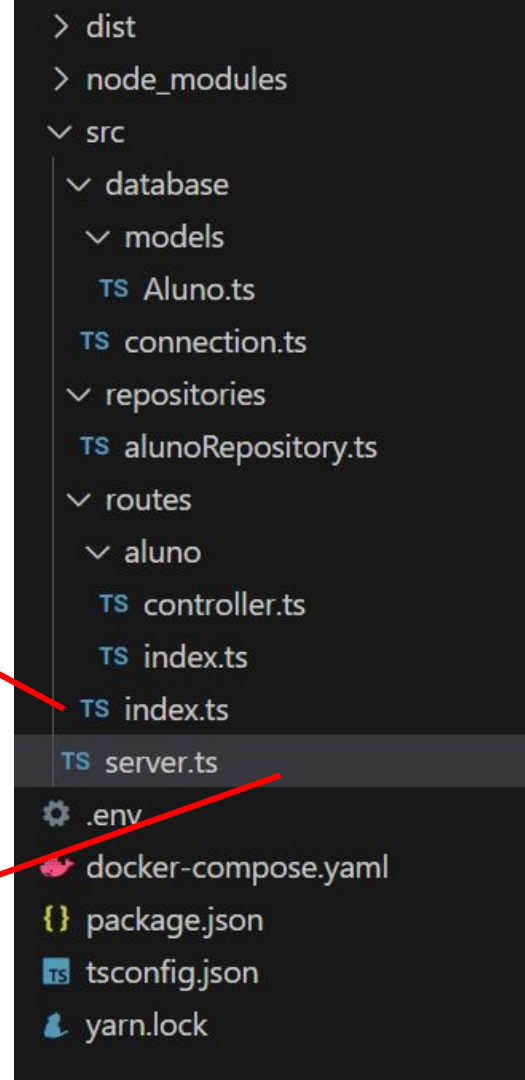
```
npm install --save sequelize
```

```
# One of the following:
$ npm install --save pg pg-hstore # Postgres
$ npm install --save mysql2
$ npm install --save mariadb
$ npm install --save sqlite3
$ npm install --save tedious # Microsoft SQL Server
$ npm install --save oracledb # Oracle Database
```

Sequelize

```ts
import express, { Router } from "express";
import aluno from "./aluno";

const v1: Router = express.Router();

v1.use("/", aluno);

export default v1;
```

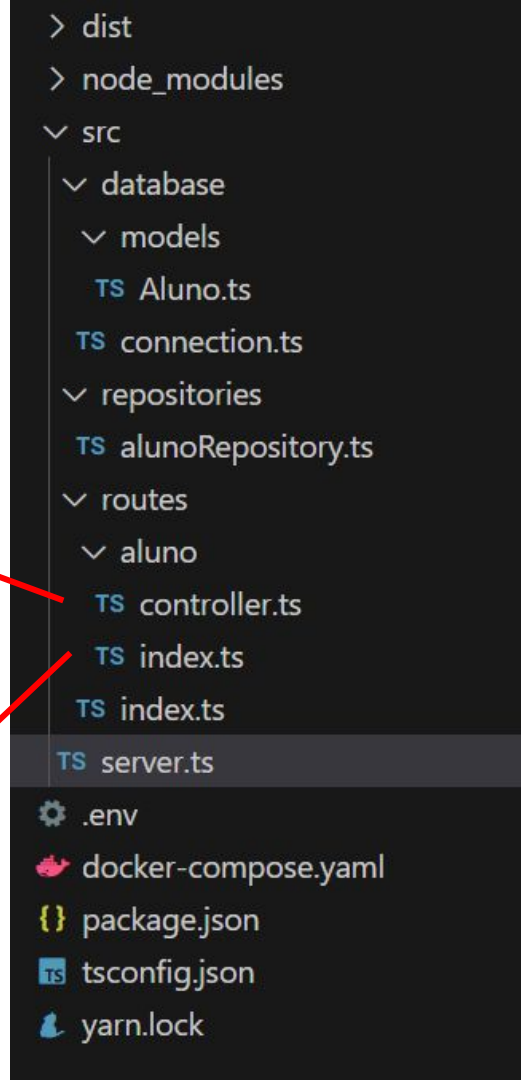
```ts
import express from "express";
import "./database/connection"
import rotas from "./routes";

const app = express();
app.use(express.json());

app.use("/v1", rotas);

app.listen(3000);
```



Sequelize

```ts
export const listarAlunos = async (
    req: Request,
    res: Response,
    next: NextFunction
) => {
    const repository = new AlunoRepository();
    let alunos = await repository.getAll();

    console.log(alunos)

    res.status(200).json({ alunos });

};
```

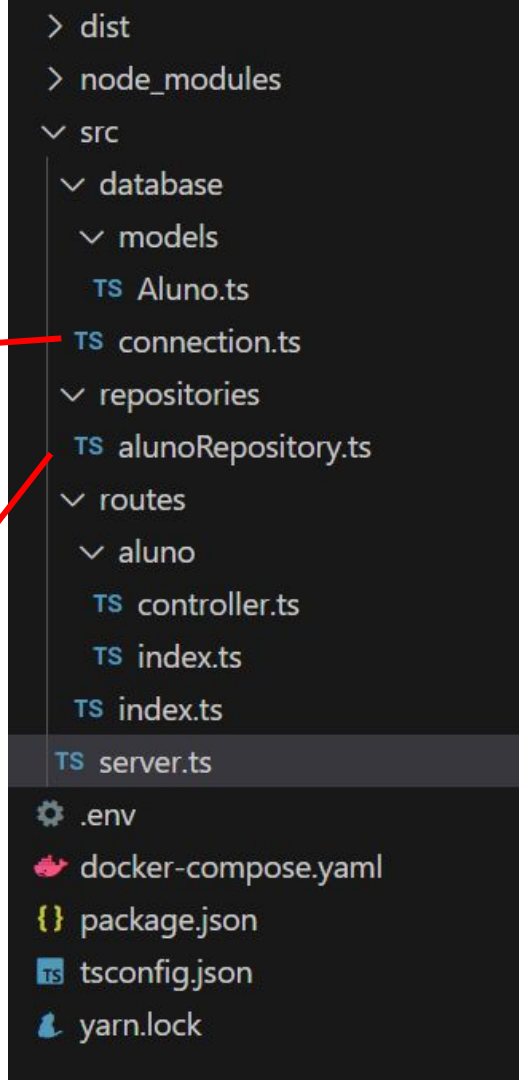
```ts
import express, { Router } from "express";
import { listarAlunos } from "./controller";

const alunos: Router = express.Router();

alunos.get("/aluno", listarAlunos);

export default alunos;
```



Sequelize

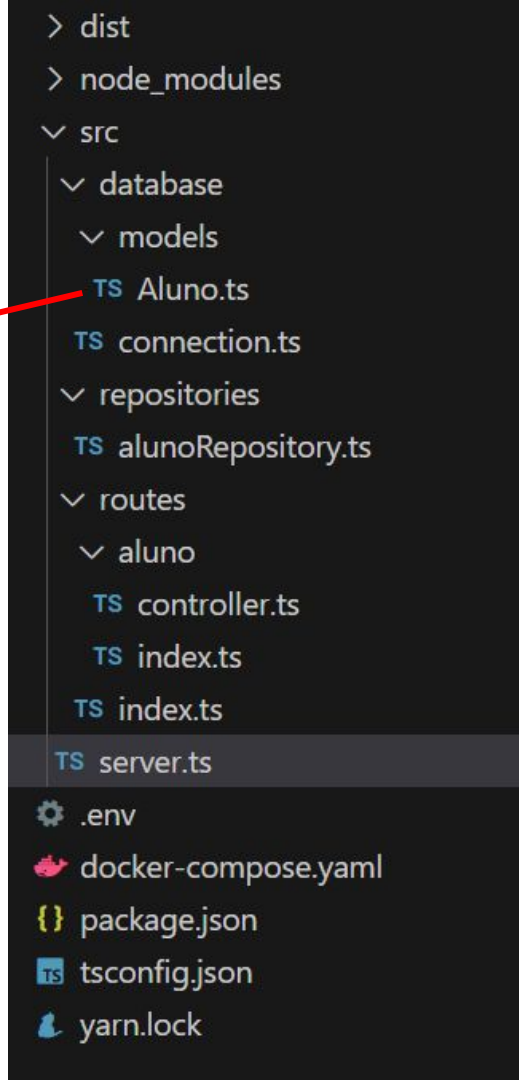
```typescript
import { Sequelize } from "sequelize-typescript";

const sequelize = new Sequelize({
  database: 'postgres',
  dialect: 'postgres',
  username: 'postgres',
  password: 'postgres',
  host: '127.0.0.1',
  port: 15432,
  models: [__dirname + "/models"],
});

export default sequelize;
```

```typescript
import Aluno from "../database/models/Aluno";

export default class AlunoRepository {

    getAll(options: Record<string, any> = {}): Promise<Array<Aluno>> {
        return Aluno.findAll();
    }

}
```



Sequelize

```ts
import {
    Table,
    Column,
    Model,
    DataType,
    CreatedAt,
    UpdatedAt,
  } from "sequelize-typescript";

  @Table({
    timestamps: true,
    tableName: "aluno",
    modelName: "Aluno",
  })
  class Aluno extends Model{
    @Column({
      primaryKey: true,
      type: DataType.BIGINT,
    })
    declare id: number;

    @Column({
      type: DataType.STRING,
    })
    declare nome: string;

    @Column({
        type: DataType.STRING,
      })
      declare matricula: string;

    @CreatedAt
    declare created_at: Date;

    @UpdatedAt
    declare updated_at: Date;

  }

  export default Aluno;
```



Sequelize

- Select
- Insert
- Update
- Delete
- Join

```
const project = await Project.findByPk(123);
if (project === null) {
  console.log('Not found!');
} else {
  console.log(project instanceof Project); // true
  // Its primary key is 123
}
```

```
const project = await Project.findOne({ where: { title: 'My Title' } });
if (project === null) {
  console.log('Not found!');
} else {
  console.log(project instanceof Project); // true
  console.log(project.title); // 'My Title'
}
```

Sequelize

- Select
- Insert
- Update
- Delete
- Join

```
const { count, rows } = await Project.findAndCountAll({
  where: {
    title: {
      [Op.like]: 'foo%'
    }
  },
  offset: 10,
  limit: 2
});
console.log(count);
console.log(rows);
```

Sequelize

- Select
- Insert
- Update
- Delete
- Join

```
const { Op } = require("sequelize");
Post.findAll({
  where: {
    [Op.or]: [
      { authorId: 12 },
      { authorId: 13 }
    ]
  }
});
```

Sequelize

- Select
- Insert
-
- Update
- Delete
- Join

```
const user = await User.create({
  username: 'alice123',
  isAdmin: true
}, { fields: ['username'] });
```

```
const captains = await Captain.bulkCreate([
  { name: 'Jack Sparrow' },
  { name: 'Davy Jones' }
]);
```

Sequelize

- Select
- Insert
- Update
- Delete
- Join

```
// Change everyone without a last name to "Doe"
await User.update({ lastName: "Doe" }, {
  where: {
    lastName: null
  }
});
```

Sequelize

- Select
- Insert
- Update
- Delete
- Join

```
// Delete everyone named "Jane"
await User.destroy({
  where: {
    firstName: "Jane"
  }
});
```

```
// Truncate the table
await User.destroy({
  truncate: true
});
```

Sequelize

- Select
- Insert
- Update
- Delete
- Join
  - HasMany
  - HasOne
  - BelongsTo
  - BelongsToMany

```
const A = sequelize.define('A', /* ... */);
const B = sequelize.define('B', /* ... */);

A.hasOne(B); // A HasOne B
A.belongsTo(B); // A BelongsTo B
A.hasMany(B); // A HasMany B
A.belongsToMany(B, { through: 'C' }); // A
```

Sequelize

- Join
  - HasMany
  - HasOne
  - BelongsTo
  - BelongsToMany

```
@Table
class Book extends Model {
  @BelongsToMany(() => Author, () => BookAuthor)
  authors: Author[];
}

@Table
class Author extends Model {
  @BelongsToMany(() => Book, () => BookAuthor)
  books: Book[];
}

@Table
class BookAuthor extends Model {
  @ForeignKey(() => Book)
  @Column
  bookId: number;

  @ForeignKey(() => Author)
  @Column
  authorId: number;
}
```

Sequelize

- Scope
  - Retornar apenas o necessários
  - Model
  - Filtros
  - Separação

```
@DefaultScope(() => ({
  attributes: ['id', 'primaryColor', 'secondaryColor', 'producedAt'],
}))
@Scopes(() => ({
  full: {
    include: [Manufacturer],
  },
  yellow: {
    where: { primaryColor: 'yellow' },
  },
}))
```

```
await Project.scope('random', { method: ['accessLevel', 19] }).findAll()
```

Sequelize

- Transaction
  - Blocos
  - Commit
  - Rollback

```
// First, we start a transaction from your
const t = await sequelize.transaction();

try {

  // Then, we do some calls passing this t

  const user = await User.create({
    firstName: 'Bart',
    lastName: 'Simpson'
  }, { transaction: t });

  await user.addSibling({
    firstName: 'Lisa',
    lastName: 'Simpson'
  }, { transaction: t });

  // If the execution reaches this line, n
  // We commit the transaction.
  await t.commit();
```

Sequelize

- Sequelize.literal
- Sequelize.where
- Sequelize.fn
- Sequelize.col
- Order: [['title', 'DESC']]
- Group
- limit
- offset

Exercícios

1. Refatore a API que está no teams para uma arquitetura em camadas
2. Modifique o acesso ao banco de dados para utilizar o Knex
   - npm install knex sqlite3
3. Crie outra versão da API utilizando o Sequelize